Process Management

Operating Systems

Universität Innsbruck

Summer Semester 2026

1 Process Memory Layout

In this exercise, you will get insight into the memory layout of a process and its different areas.

1.1 The memory map

Compile and run the following code. It contains a label `foo`, and if everything works fine, you should see the address of the `foo` label printed in the terminal. The program will then hang, waiting for keyboard input.

```c
#include <stdio.h>
#include <unistd.h>

int main() {
   foo:
    printf("the code: %p\n", &&foo);
    pause();
    return 0;
}
```

So we have the address of the `foo` label; it could look like `0x40052a` (or `0x55d8ef4426ce`, depending on which version of Linux you are running). Hit any key to allow the program terminate, and then we try something else. We will now tell the shell to start the process in the background. That will allow us to use the shell while the program is suspended, waiting for input.

```
./01_memory_map &
[1] 2708
the code: 0x55968c40f6ce
```

The number `2708` (or whatever you see) is the process identifier. Now we take a look at the file `/proc/#id/maps`. Take a look at it using the cat command.

```
cat /proc/2708/maps
:
```

This file contains the process's memory mapping. Locate the code segment. Does the address of label `foo` correspond to it? To bring the suspended process back to the foreground, you use the `fg` command. Then you can press Ctrl+C to terminate the program.

```
fg
./01_memory_map
^C
```

Now we want to see where the global variables are stored in memory. The following code will get its own process identifier and print the content of `proc/<pid>/maps` using a library procedure `system()` that will read a command from a string and then execute it in a sub-process:

```c
#include <stdlib.h>
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

char global[] = "This is a global string";
int main() {
    int pid = getpid();
   foo:
    printf("process id: %d\n", pid);
    printf("global string: %p\n", &global);
    printf("the code: %p\n", &&foo);
    printf("\n\n /proc/%d/maps \n\n", pid);
    char command[50];
    sprintf(command, "cat /proc/%d/maps", pid);
    system(command);
    return 0;
}
```

It will print the address of the global variable `global`. In which segment do you find it?

1.2 Stack

The stack in C/Linux is located almost at the top of the user space and grows downward. You will find it in the process map, and the entry will probably look something like this:

```
7ffed89d4000-7ffed89f5000 rw-p 00000000 00:00 0      [stack]
```

1.2.1 Simple stack

Now we want to see how a local variable inside main is located in the memory with the following code:

```c
#include <stdlib.h>
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

int main() {
    int pid = getpid();
    unsigned long p = 0x1;
    printf("p (0x%lx): %p \n", p, &p);
    printf("\n\n /proc/%d/maps \n\n", pid);
    char command[50];
    sprintf(command, "cat /proc/%d/maps", pid);
    system(command);
    return 0;
}
```

Execute the program and verify that `p`'s location is in the stack segment.

1.2.2 Pushing into stack

Now we make some procedure calls and see how the stack grows.

```
#include <stdlib.h>
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

void zot(unsigned long *stop) {
    unsigned long r = 0x3;
    unsigned long *i;
    for(i = &r; i <= stop; i++)
        printf("%p 0x%lx\n", i, *i);
}

void foo(unsigned long *stop ) {
    unsigned long q = 0x2;
    zot(stop);
}

int main() {
    int pid = getpid();
    unsigned long p = 0x1;
    foo(&p);
   back:
    printf(" p: %p \n", &p);
    printf(" back: %p \n", &&back);
    printf("\n\n /proc/%d/maps \n\n", pid);
    char command[50];
    sprintf(command, "cat /proc/%d/maps", pid);
    system(command);
    return 0;
}
```

If this works, you should have a nice stack trace. Explain what you observe based on the code.

1.3 Heap

The heap is used for data structures created dynamically at runtime. It is needed when we have data structures whose size is known only at runtime, or that must survive the return of a procedure call. When `malloc()` is called, it allocates memory on the heap. The following code represents how a heap variable is located in memory.

```
#include <stdlib.h>
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

int main() {
    int pid = getpid();
    char *heap = malloc(20);
```

```
    printf("the heap variable at: %p\n", &heap);
    printf("pointing to: %p\n", heap);
    printf("\n\n /proc/%d/maps \n\n", pid);}
    char command[50];
    sprintf(command, "cat /proc/%d/maps", pid);
    system(command);
    return 0;
}
```

Explain how it corresponds to the heap memory address in maps (considering where the heap variable is stored and where it is pointing at).

1.3.1 Free and reuse

The problem with using the heap is not how to allocate memory but to free it. You should free it only once and not reuse the freed memory. The compiler will detect some apparent errors, but most errors will only appear at runtime and might be very hard to track down. If you allocate a data structure of 20 bytes every millisecond and forget to free it, you might run out of memory. In the following code, we will see how reusing a freed pointer will cause a problem.

```
#include <stdlib.h>
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

int main() {
    char *heap = malloc(20);
    *heap = 0x61;
    printf("heap pointing to: 0x%x\n", *heap);
    free(heap);
    char *foo = malloc(20);
    *foo = 0x62;
    printf("foo pointing to: 0x%x\n", *foo);
    /* danger ahead */
    *heap = 0x63;
    printf("or is it pointing to: 0x%x\n", *foo);
    return 0;
}
```

Explain what happens here and what the problem is.

1.4 Shared libraries

When you printed out the memory details of a process in the `maps` file, in addition to code, data, strange kernel stuff, the stack, and the heap, there were some memory lines between heap and stack. Some are executable, some writable, and some are like:

```
/lib/x86_64-linux-gnu/ld-2.23.so
```

All the libraries required for printing messages, finding the process identifier, allocating memory, etc are located somewhere in these segments.

2 Process Creation (Parent and Child Processes)

2.1 fork()

In this exercise, we will explore how processes are created. We will use the library procedure `fork()`, which you can learn about with:

```
man fork
```

The `fork` function requires the `unistd.h` library, so we need to include it. It returns both the child process identifier and zero. Execute the following code and explain what is happening.

```c
#include <stdio.h>
#include <unistd.h>

int main() {
    int pid = fork();
    printf("pid = %d\n", pid);
    return 0;
}
```

Calling `fork()` somehow creates a duplicate of the executing process, and the execution then continues in both copies. By checking the returned value, we can determine whether we are executing in the parent process or in the child process, and execute the appropriate code for each. Try this extension to the program and explain what happens.

```c
#include <stdio.h>
#include <unistd.h>

int main() {
    int pid = fork();
    if (pid == 0) { // child
        printf("I'm the child process and my pid is %d.\n", getpid());
        sleep(1);
    } else { // parent
        printf("I'm the parent process and my child is called %d.\n", pid);
    }
    printf("This is the end of %d!\n", getpid());
    return 0;
}
```

2.2 wait() and waitpid()

To terminate the program in a more controlled way, we can have the parent `wait()` for the child process to terminate. Execute the following code to see this mechanism.

```c
int main() {
    int pid = fork();
    if(pid == 0) {
        printf("I'm the child %d\n", getpid());
        sleep(1);
```

```
    } else {
        printf("My child is called %d\n", pid);
        wait(NULL);
        printf("My child has terminated \n");
    }
    printf("This is the end (%d)\n", getpid());
    return 0;
}
```

In this program parent process will wait for all of its children to terminate. Now we want the parent to only wait for a specific child. See the use of `waitpid()` in the following code:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();
    if (pid == 0) {
        // Child
        printf("I'm the child %d\n", getpid());
        sleep(1);
        return 42;
    } else {
        // Parent
        int status;
        printf("My child is called %d\n", pid);
        waitpid(pid, &status, 0);
        // wait for THIS child only
        if (WIFEXITED(status)) {
            printf("My child exited with status %d\n",
            WEXITSTATUS(status));
        }
        printf("My child has terminated\n");
    }
    printf("This is the end (%d)\n", getpid());
    return 0;
}
```

Question 1: Change the above code to create two children and force the parent to wait for the first child.

2.3 Returning a value

A process can produce a value (e.g., an integer) when it terminates, and the parent process can pick up this value. If we change the program so that the child process returns 42 as it exists, the value can be picked up using the `wait()` procedure.

```
int main() {
    int pid = fork();
    if(pid == 0)
        return 42;
    else {
```

```
        int res;
        wait(&res);
        printf("the result was %d\n", WEXITSTATUS(res));
    }
    return 0;
}
```

2.4 exec()

The `exec` functions replace the currently running process (the child)'s memory space with a new program, overwriting its code, data, heap, and stack while keeping the same process ID. Here is a family of `exec()` functions, all of which have slightly different characteristics:

```
int execl (const char *path, const char *arg, ...);
int execlp(const char *file, const char *arg, ...);
int execle(const char *path, const char *arg, ..., char *const envp[]);
int execv (const char *path, char *const argv[]);
int execvp(const char *file, char *const argv[]);
int execve(const char *file, char *const argv[], char *const envp[]);
```

> Each system call is the word `exec` followed by either `l` or `v`, and then possibly followed by either `e` or `p`.
>
> If the name ends in either `l` or `v`, the program name must be given in full.
>
> If the name is appended by a `p`, it will search for the file using the current environment variable `PATH`, which usually includes `/bin/`, `/usr/bin/`, etc.

We use the following code to execute `ls` command inside child process:

```
int main() {
    int pid = fork();
    if(pid == 0) {
        execlp("ls", "ls", NULL);
        printf("this will only happen if exec fails\n");
    } else {
        wait(NULL);
        printf("we're done\n");
    }
    return 0;
}
```

The call to `execlp()` will locate the program `ls` and then replace the process's code and data segments with those from the executable binary. The stack and heap areas are reset, so the program starts execution from scratch.

2.5 exit() and abort()

We can terminate the process normally with `exit()`, which terminates the calling process. In the following example child process terminates with `exit()`.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    printf("Start (pid=%d)\n", getpid());
    pid_t pid = fork();
    if (pid == 0) {
        // Child
        printf("Child (pid=%d): exiting\n", getpid());
        exit(0);
    } else {
        // Parent
        wait(NULL);
        printf("Parent (pid=%d): child finished\n", getpid());
    }
    printf("End (pid=%d)\n", getpid());
    return 0;
}
```

Abort terminates a process abnormally by sending the signal `SIGABRT` to the calling process:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();
    if (pid == 0) {
        // Child
        printf("Child (%d): aborting!\n", getpid());
        abort();  // sends SIGABRT
    } else {
        // Parent
        int status;
        waitpid(pid, &status, 0);
        if (WIFSIGNALED(status)) {
            printf("Parent: child killed by signal %d\n", WTERMSIG(status));
        }
    }
    return 0;
}
```

2.6 Zombie

A zombie process is a process that has already terminated but whose parent process has not yet collected its termination status. Until the parent calls `wait()`, the operating system keeps a minimal entry for the child process in the process table. This retained information allows the parent to retrieve the child's exit status and return value. If the system were to remove the

child process immediately, this information would be lost. We can observe this behavior when a child process terminates, and the parent deliberately delays calling `wait()`.

To see the behavior of a Zombie compile and run the following program in the background.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();
    if(pid == 0) {
        printf("check the status\n");
        sleep(10);
        printf("and again\n");
        return 42;
    } else {
        sleep(20);
        int res;
        wait(&res);
        printf("the result was %d\n", WEXITSTATUS(res));
        printf("and again\n");
        sleep(10);
    }
    return 0;
}
```

Check the status of the processes using the `ps` command. Notice how the two processes are created, how the child becomes a zombie, and how it is removed from the system once we receive the return value.

```
./15_zombie&
:
ps -ao pid,stat,command
:
```

2.7 Orphans

In the following code, we attempt to crash the parent process and observe the outcome.

```
int main() {
    int pid = fork();
    if(pid == 0) {
        int child = getpid();
        printf("child: parent %d, group %d\n", getppid(), getpgid(child));
        sleep(4);
        printf("child: parent %d, group %d\n", getppid(), getpgid(child));
        sleep(4);
        printf("child: parent %d, group %d\n", getppid(), getpgid(child));
    } else {
        int parent = getpid();
```

```
        printf("parent: parent %d, group %d\n", getppid(), getpgid(parent));
        sleep(2);
        int zero = 0;
        int i = 3 / zero;
    }
    return 0;
}
```

Compile and run the program to observe how the child process's parent identifier changes. The process has become an orphan and has been adopted by the `upstart` process (or `init` or `systemd`, depending on the system you are using). Note the new process identifier and then check its state using the `ps` command:

```
ps <whatever the process id was>
:
```

Also, look at the process tree:

```
pstree <whatever the process id was>
:
```